

LangChain & Agentic AI – From Basics to Advanced 23

RunnableSequence — Intermediate Values, Batch Processing & Revision

1. How to Get Intermediate Values from a Chain?

Consider the following RunnableSequence.

```
from langchain_core.runnables import RunnableLambda
```

```
add_ten = RunnableLambda(  
    lambda x: x + 10  
)
```

```
multiply_two = RunnableLambda(  
    lambda x: x * 2  
)
```

```
subtract_five = RunnableLambda(  
    lambda x: x - 5  
)
```

```
chain = (  
    add_ten  
    | multiply_two  
    | subtract_five  
)
```

```
print(type(chain))
```

```
result = chain.invoke(5)
```

```
print(result)
```

Execution Flow

5

↓

add_ten

↓

15

↓

multiply_two

↓

30

↓

subtract_five

↓

25

Final result:

25

But sometimes we may want to access:

After add → 15

After multiply → 30

After subtract → 25

These are called **intermediate values**.

2. Approach 1 — Store Intermediate Values in a Dictionary

We can use a Python dictionary to save the output after every step.

```
from langchain_core.runnables import RunnableLambda
```

```
values = {}
```

```
def add_ten(x):  
    result = x + 10  
    values["after_add"] = result  
    return result
```

```
def multiply_two(x):  
    result = x * 2  
    values["after_multiply"] = result  
    return result
```

```
def subtract_five(x):  
    result = x - 5  
    values["after_subtract"] = result  
    return result
```

```
add = RunnableLambda(add_ten)  
multiply = RunnableLambda(multiply_two)  
subtract = RunnableLambda(subtract_five)
```

```
chain = add | multiply | subtract
```

```
result = chain.invoke(5)
```

```
print("Final Result   :", result)
```

```
print("After Add      :", values["after_add"])
```

```
print("After Multiply :", values["after_multiply"])
```

```
print("After Subtract :", values["after_subtract"])
```

Output

Final Result : 25

After Add : 15

After Multiply : 30

After Subtract : 25

Flow

5

↓

add_ten

↓

15 —————→ values["after_add"]

↓

multiply_two

↓

30 —————→ values["after_multiply"]

↓

subtract_five

↓

25 —————→ values["after_subtract"]

↓

Final Result

Important Point

Every function does two jobs:

1. Performs its calculation
2. Saves the result in the values dictionary

But it must still:

return result

because that result has to become the input to the next Runnable.

3. Approach 2 — Create a Common save() Function

Instead of writing the saving logic in every function, we can create one common function.

```
from langchain_core.runnables import RunnableLambda
```

```
values = {}
```

```
def save(name, value):
```

```
    values[name] = value
```

```
    return value
```

```
add_ten = RunnableLambda(
```

```
    lambda x: save("after_add", x + 10)
```

```
)
```

```
multiply_two = RunnableLambda(
```

```
    lambda x: save("after_multiply", x * 2)
```

```
)
```

```
subtract_five = RunnableLambda(
```

```
lambda x: save("after_subtract", x - 5)
)
```

```
chain = add_ten | multiply_two | subtract_five
```

```
result = chain.invoke(5)
```

```
print("Final :", result)
```

```
print(values)
```

Output

Final : 25

```
{
  'after_add': 15,
  'after_multiply': 30,
  'after_subtract': 25
}
```

4. Understanding the save() Function

Consider:

```
def save(name, value):  
    values[name] = value  
    return value
```

Suppose we call:

```
save("after_add", 15)
```

It performs:

```
values["after_add"] = 15
```

and then returns:

15

Therefore, the value is:

Saved

+

Passed to the next Runnable

This is important in a sequence.

5. Complete Intermediate Value Flow

Input = 5

↓

add_ten



save("after_add", 15)



Dictionary stores 15



Returns 15



multiply_two



save("after_multiply", 30)



Dictionary stores 30



Returns 30



subtract_five



save("after_subtract", 25)



Dictionary stores 25



Returns 25



Final Result = 25

6. Another LangChain Approach

LangChain also provides:

RunnablePassthrough

It can be useful when we want to pass existing data forward while adding or preserving intermediate information in the chain.

For simple learning programs, using:

Dictionary

+

RunnableLambda

is also an easy way to understand intermediate values.

7. Multi-Step LLM RunnableSequence

Now consider a real AI workflow.

Requirement

Topic

↓

Generate Explanation

↓

Use Explanation

↓

Generate MCQs

This is a sequential workflow because the second task depends on the result of the first task.

8. Complete Multi-Step LLM Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.runnables import RunnableLambda
```

```
model1 = ChatOpenAI(  
    model="gpt-5.6-luna",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
model2 = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
parser = StrOutputParser()
```

```
explanation_prompt = PromptTemplate.from_template(
    """
    Explain {topic} for a beginner.

    Use very simple English.
    Give one real-world example.
    """
)
```

)

```
prepare_quiz_input = RunnableLambda(
```

```
    lambda content: {
```

```
        "content": content
```

```
    }
```

```
)
```

```
quiz_prompt = PromptTemplate.from_template(
```

```
    """
```

```
    Based only on the following content,  
    generate 5 MCQs.
```

```
    Content:
```

```
    {content}
```

```
    """
```

```
)
```

```
chain = (  
    explanation_prompt  
    | model1  
    | parser  
    | prepare_quiz_input  
    | quiz_prompt  
    | model2  
    | parser  
)
```

```
responses = chain.batch(  
    [  
        {"topic": "Java"},  
        {"topic": "Python"},  
        {"topic": "LangChain"}  
    ]  
)
```

```
for response in responses:
```

```
    print(response)
```

```
    print(
```

```
"\n=====
====="
```

```
)
```

9. Understanding This Chain

The chain is:

explanation_prompt



model1



parser



prepare_quiz_input



quiz_prompt



model2



parser

For one topic:

```
{"topic": "Java"}
```



Explanation Prompt



Model 1



Parser



Explanation String



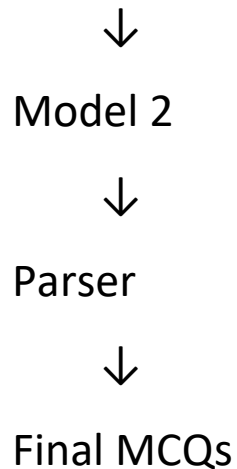
RunnableLambda



```
{"content": explanation}
```



Quiz Prompt



10. Why Do We Need `prepare_quiz_input`?

After:

`explanation_prompt | model1 | parser`

the parser produces a string.

Example:

Java is a programming language...

But the next prompt expects:

`{content}`

Therefore, we convert the string into a dictionary.

```
prepare_quiz_input = RunnableLambda(  
    lambda content: {  
        "content": content  
    }  
)
```

)

It converts:

Java is a programming language...

into:

{

 "content": "Java is a programming language..."

}

Now the data is suitable for:

quiz_prompt

11. Input/Output Compatibility

One of the most important rules in RunnableSequence is:

The output of the previous Runnable should be compatible with the input expected by the next Runnable.

Concept:

Runnable A



Output



Must be suitable for



Runnable B Input

If the formats are not compatible, we can use something like:

RunnableLambda

as an adapter.

12. How batch() Works in This Program

We are passing:

```
[  
    {"topic": "Java"},  
    {"topic": "Python"},  
    {"topic": "LangChain"}  
]
```

to:

`chain.batch(...)`

Each input passes through the **whole sequence**.

Java

Java



Whole RunnableSequence



Java MCQs

Python

Python



Whole RunnableSequence



Python MCQs

LangChain

LangChain



Whole RunnableSequence



LangChain MCQs

So:

Multiple Inputs



Same RunnableSequence



Multiple Outputs

13. Does RunnableSequence Mean Multiple LLMs?

No.

RunnableSequence means:

Multiple Runnables execute one after another.

The sequence may contain:

Prompt

Model

Parser

RunnableLambda

or many other Runnable components.

Example:

add_ten | multiply_two | subtract_five

There is no LLM here.

But it is still a sequential Runnable workflow.

14. Can RunnableSequence Contain Multiple LLM Calls?

Yes.

Example:

Prompt 1

↓

Model

↓

Parser

↓

Prompt 2

↓

Model

↓

Parser

This sequence contains:

Model Call 1

+

Model Call 2

Therefore:

One `chain.invoke()`

does not necessarily mean:

One Model API Call

15. Too Many Model Calls

Suppose:

Prompt 1 → Model

Prompt 2 → Model

Prompt 3 → Model

We may have three model calls.

More model calls can result in:

More Tokens

+

More Cost

+

More Latency

Therefore, multi-step workflows should be created when the additional steps are really required.

16. Common Mistakes

Mistake 1 — Thinking Pipe and RunnableSequence Are Different Workflows

prompt | model | parser

commonly creates a sequential composition represented as a:
RunnableSequence

Mistake 2 — Wrong Order of Runnable Components

Wrong:

parser | model | prompt

The order should match the required data flow.

Example:

prompt | model | parser

Mistake 3 — Ignoring Input/Output Compatibility

Always check:

Output of Previous Step



Is it suitable for



Input of Next Step?

Mistake 4 — Thinking Every Step Gets the Original Input

In a normal sequence:

$A \rightarrow B \rightarrow C$

normally:

A receives initial input

B receives A's output

C receives B's output

Mistake 5 — Too Many Model Calls

Too many model calls can increase:

Cost

+

Latency

Mistake 6 — Thinking RunnableSequence Means Multiple LLMs

RunnableSequence means:

Multiple Runnables

executed

one after another

It does not mean multiple LLMs are compulsory.

17. MCQs

MCQ 1

What is RunnableSequence?

- A. A vector database
- B. Runnables executed one after another
- C. An API key
- D. Conversation memory

Answer: B

MCQ 2

In a sequence:

$A \rightarrow B \rightarrow C$

what does B normally receive?

- A. Only original input
- B. A's output
- C. C's output
- D. Nothing

Answer: B

MCQ 3

What does this commonly create?

prompt | model | parser

- A. RunnableParallel
- B. RunnableSequence
- C. Vector Store
- D. Agent

Answer: B

MCQ 4

Can we create RunnableSequence explicitly?

- A. Yes
- B. No

Answer: A

MCQ 5

Which statement is most important?

- A. Every Runnable gets original input
- B. Every step's output normally becomes the next step's input
- C. RunnableSequence requires three LLMs
- D. RunnableSequence cannot use invoke()

Answer: B

MCQ 6

What is RunnableLambda useful for?

- A. Convert a Python callable into a Runnable
- B. Create OpenAI API keys
- C. Store embeddings
- D. Replace LangChain

Answer: A

MCQ 7

Suppose Step A returns a string but Step B expects a dictionary. What should we think about?

- A. Input/output compatibility
- B. Vector similarity
- C. API key security
- D. Context window

Answer: A

MCQ 8

Which workflow is naturally sequential?

A

Generate Outline



Write Article Using Outline

B

Generate Tweet

Generate Blog

Generate Headline

Answer: A

The article depends on the outline, so there is a dependency.

18. Important Points

1. Sequential execution means operations run one after another.
2. `RunnableSequence` executes `Runnables` sequentially.
3. Output of one `Runnable` becomes input to the next `Runnable`.
4. The following creates sequential composition:

`prompt | model | parser`

5. This composition is represented as:

`RunnableSequence`

6. We can also create one explicitly:

`RunnableSequence(`

`prompt,`

```
model,  
parser  
)
```

7. Pipe syntax is normally simpler:

```
prompt | model | parser
```

8. Order matters.

9. Input/output compatibility between steps matters.

10. RunnableLambda can convert a normal Python callable into a Runnable.

11. RunnableLambda can also be useful as a small adapter between incompatible steps.

12. A sequence can contain multiple model calls.

13. One:

```
chain.invoke()
```

does not necessarily mean only one model API call.

14. Sequential workflows are best when later tasks depend on earlier results.

19. Dependency Is the Key

This is the most important point to remember.

When Tasks Are Dependent

Example:

Generate Outline



Write Blog Using Outline

The second task requires the output of the first task.

Therefore:

DEPENDENCY



RunnableSequence

20. When Tasks Are Independent

Suppose we want:

$r \rightarrow$ Generate Tweet

Topic $\longrightarrow \perp \rightarrow$ Generate Blog

$L \rightarrow$ Generate Headline

These tasks do not necessarily depend on each other's output.

Therefore:

NO DEPENDENCY



Independent Tasks



RunnableParallel

21. RunnableSequence vs RunnableParallel

RunnableSequence

Tasks are dependent

One task follows another

Previous output may be required

Order matters

$A \rightarrow B \rightarrow C$

RunnableParallel

Tasks are independent

Tasks can execute independently

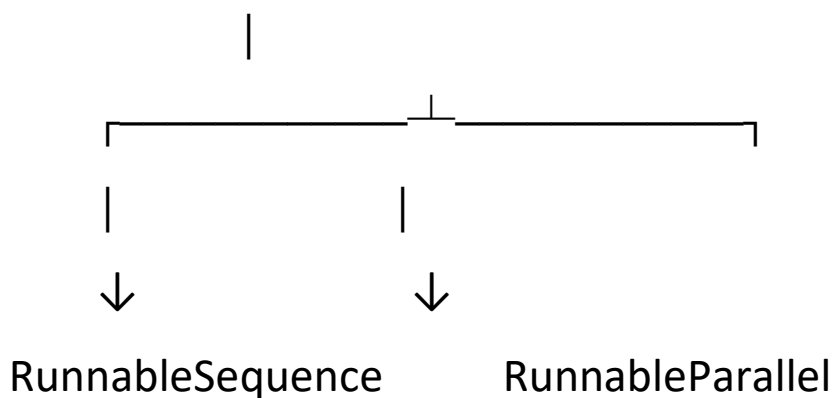
Previous task output is not required

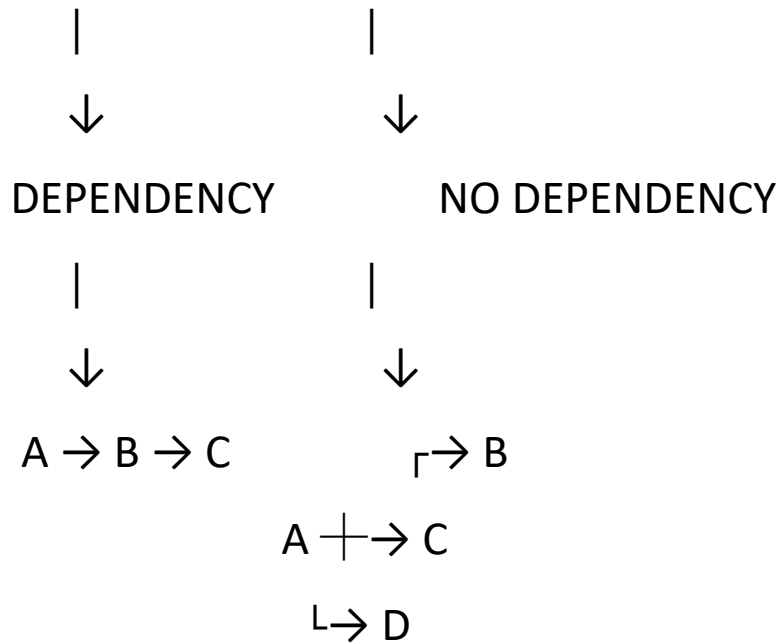
Sequence between independent tasks is not the main requirement

$A \rightarrow B, C, D$

22. Master Diagram

RUNNABLE WORKFLOWS





23. Final Memory Formula

RunnableSequence

Input



Runnable A



Output A



Runnable B



Output B



Runnable C



Final Output

Remember:

Output of Left



Input of Right

24. One-Line Conclusion

Use `RunnableSequence` when tasks have dependency and must execute one after another. Use `RunnableParallel` when tasks are independent and there is no dependency between them.

Easy Memory

DEPENDENCY



SEQUENCE

NO DEPENDENCY



PARALLEL

